

(/)

Post

Follow @caervs

Hello World, Byte for Byte

An Intro to Linux Binaries (30 - 40 mins read time)

Have you ever wondered how your OS runs a program? How it takes that incomprehensible sequence of binary in `/bin/bash` or `/usr/local/bin/node` and turns it into a shell? Or a web service? Today we're going to start to answer that question by taking a deep look at the x86 (<https://en.wikipedia.org/wiki/X86>) binary for the classic 'Hello, world!' program (https://en.wikipedia.org/wiki/%22Hello,_World!%22_program).

```
$ ./hello_world
Hello, world!
$ od -x --endian=big hello_world
00000000 7f45 4c46 0201 0100 0000 0000 0000 0000
00000020 0200 3e00 0100 0000 0010 4000 0000 0000
00000040 4000 0000 0000 0000 5021 0000 0000 0000
00000060 0000 0000 4000 3800 0300 4000 0600 0500
00000100 0100 0000 0400 0000 0000 0000 0000 0000
00000120 0000 4000 0000 0000 0000 4000 0000 0000
00000140 e800 0000 0000 0000 e800 0000 0000 0000
00000160 0010 0000 0000 0000 0100 0000 0500 0000
00000200 0010 0000 0000 0000 0010 4000 0000 0000
00000220 0010 4000 0000 0000 2200 0000 0000 0000
00000240 2200 0000 0000 0000 0010 0000 0000 0000
00000260 0100 0000 0600 0000 0020 0000 0000 0000
00000300 0020 4000 0000 0000 0020 4000 0000 0000
00000320 0e00 0000 0000 0000 0e00 0000 0000 0000
00000340 0010 0000 0000 0000 0000 0000 0000 0000
00000360 0000 0000 0000 0000 0000 0000 0000 0000
*
...
```

Maybe this looks like a mystifying incantation of numbers now, but by the end you'll understand every last one of those bytes and have the tools to understand any other executable binary. As is our fashion here on The Lambda Scheme we won't be content just listing the facts, rather we'll also try to answer a lot of the *why*'s. Why? Why do programs look this way? Why do they run this way?

To that end we'll start with some foundational topics in Part 1 exploring the functions of stored program computing, segmented memory, and how the Linux kernel executes with special privileges. In Part 2 we'll see in detail how the ELF

(https://en.wikipedia.org/wiki/Executable_and_Linkable_Format) binary format implements these three

ideas and do a byte-for-byte account of our *hello world* program. Finally as a bonus in Part 3 we'll take what we know and try to make our program be bootable so we can print "Hello, world!" without an operating system running. Let's do it!

Part 1: Fundamentals of Executable Code

Stored Program Computing

As we discussed in Quantum Supremacy ([/QS.html#A-Modern-Computer-Architecture](#)), the most common way to program a computer today is based on the von Neumann model (https://en.wikipedia.org/wiki/Von_Neumann_architecture) of stored-program (https://en.wikipedia.org/wiki/Stored-program_computer) instruction-based (https://en.wikipedia.org/wiki/Instruction_set_architecture) computing. This approach has become so pervasive that it's easy to forget there are alternatives. Here "instruction based" means that our program is in the form of binary-encoded instructions (https://en.wikipedia.org/wiki/Instruction_set_architecture) to be run sequentially (contrast with NISC (https://en.wikipedia.org/wiki/No_instruction_set_computing) or analog computers (https://en.wikipedia.org/wiki/Analog_computer)) and "stored program" means that we keep our program as a file on disk right next to all of the other data on our computer.

I'd love to do a post exploring how the stored program model emerged from earlier punched cards (https://en.wikipedia.org/wiki/Punched_card), plug boards (https://en.wikipedia.org/wiki/Patch_panel), and switch banks (<https://en.wikipedia.org/wiki/ENIAC>) (soon!), but here we'll discuss some of the advantages of stored program computing and how they shape the format of executable binaries.

Let's start with libraries. We're all familiar with these. Libraries let us chunk all the code responsible for related functions into the same bounded area, like a package or module, so we can reason about our system better. As such they also let us write each functional area once (https://en.wikipedia.org/wiki/Don%27t_repeat_yourself), or better yet let domain experts write it once, and compose our modules into a program we can run with confidence.

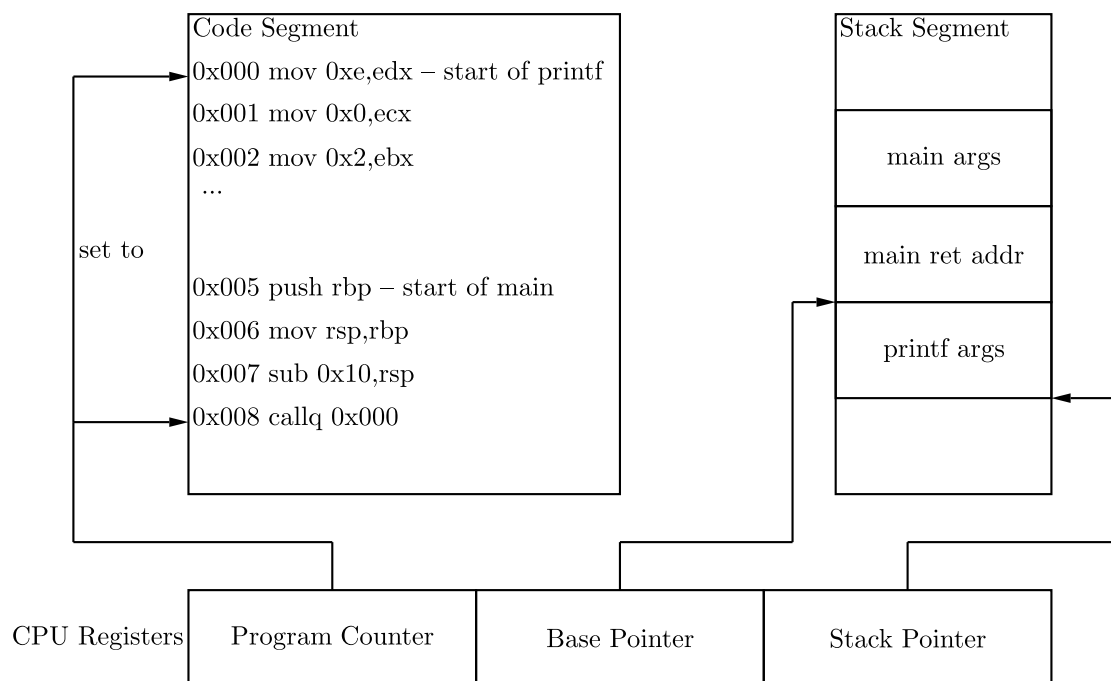
As a simple example we might write our program in C taking advantage of the standard GNU C library (<https://www.gnu.org/software/libc/>) to give us a `printf()` function

```
// helloc.c

#include <stdio.h>

int main(int argc, char **argv) {
    printf("Hello, world!\n");
}
```

Each of the functions in our program, like `main()` and `printf()`, will ultimately live at some address in memory when the program is running. What it means to call the function is to set up a stack frame (https://en.wikipedia.org/wiki/Call_stack#STACK-FRAME) for the call, push the return address (https://en.wikipedia.org/wiki/Program_counter) onto a stack, and then tell the processor to jump (https://en.wikipedia.org/wiki/Branch_(computer_science)) to that function's address in memory and start executing from there. x86 conveniently has a single instruction `call` that can do the latter two things for you (undone by returning with `ret`).



Now think for a minute what it takes to get this all working. Your computer has to lay out all of the program's functions and figure out the address at which each one will live. It then has to look for all the places in which each function is called and make the call point to the right address.

In the long long ago you would have to do this work manually. Your "libraries" would be a batch of punch cards or toggle states in some file cabinet somewhere and would have well known addresses for their included functions (no overlap with other libraries allowed). If

any procedure was rewritten its length would change as would all addresses that followed it and you would have to update all the programs that referenced any of them. God help you!

But with our programs stored alongside the rest of our data in our computer we can automate all of this work. Programs written in C or Python simply become more data for your computer to munge to create a running process. You likely have lots of experience with the first stage of this munging, compiling your source code

(<https://en.wikipedia.org/wiki/Compiler>), the result of which is assembly

(https://en.wikipedia.org/wiki/Assembly_language), so called because assembly functions are still referenced by name and an assembler can then go and **assemble** them together and resolve each name to an address. Let's see this in an example assembly file

```
# example.s

.text

func1:
    ret

.global func2
func2:
    call func1
    ret
```

where func2 calls func1. If we assemble this file into an ELF object and look at the contents we'll see our call now goes to address 0, the address of func1.

```
$ gcc -c example.s -o example.o
$ objdump -d example.o
```

```
example.o:      file format elf64-x86-64
```

```
Disassembly of section .text:
```

```
0000000000000000 <func1>:
  0:   c3                               retq

0000000000000001 <func2>:
  1:   e8 fa ff ff ff                 callq  0 <func1>
  6:   c3                               retq
```

but the assembly doesn't end there. We can actually take multiple elf objects and compose them into a single executable binary. In practice we usually have (at least) one object file per library so that we only have to pay the cost of compiling when we change the source code of that library. When we compose all our object files together it's called linking ([https://en.wikipedia.org/wiki/Linker_\(computing\)](https://en.wikipedia.org/wiki/Linker_(computing))) but it's in principle the same thing as assembly except we resolve global names instead of local ones. Let's write a separate assembly file that calls the `func2` function from our `example.s` above.

```
# example2.s

.text

.global func3
func3:
    call func2
```

Now if we assemble this one we can produce an object file that when linked with `example.o` can initiate the call sequence `func3 -> func2 -> func1`.

```
$ gcc -c example2.s -o example2.o
$ ld --entry func3 example*.o -o example
$ objdump -d example
```

```
example:      file format elf64-x86-64
```

Disassembly of section `.text`:

```
0000000000401000 <func1>:
  401000:          c3                      retq

0000000000401001 <func2>:
  401001:      e8 fa ff ff ff             callq  401000 <func1>
  401006:          c3                      retq

0000000000401007 <func3>:
  401007:      e8 f5 ff ff ff             callq  401001 <func2>
```

How does the linker know to resolve the call in `example2.s` to the right address? Two things. Firstly when you assemble a file with an unknown label, `gcc` will store that label in the object file's relocation table ([https://en.wikipedia.org/wiki/Relocation_\(computing\)](https://en.wikipedia.org/wiki/Relocation_(computing))), for future reference by the linker. Looking at the relocation table for `example2.o` we see

```
$ readelf --relocs example2.o
```

Relocation section '.rela.text' at offset 0xe8 contains 1 entry:

Offset	Info	Type	Sym. Value	Sym. Name + Addre
00000000000001	0005000000004	R_X86_64_PLT32	0000000000000000	func2 - 4

telling the linker to change the byte at offset 1 to the final address of func2 (minus 4 because the processor always adds four on the next cycle before executing). Ok, so how then does the linker know where func2 is? Whelp, similarly, example.o has a symbol table (https://en.wikipedia.org/wiki/Symbol_table) which tells the linker how to map a label to an offset.

```
$ readelf --symbols example.o
```

Symbol table '.symtab' contains 6 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000000000	0	SECTION	LOCAL	DEFAULT	1	
2:	0000000000000000	0	SECTION	LOCAL	DEFAULT	2	
3:	0000000000000000	0	SECTION	LOCAL	DEFAULT	3	
4:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	1	func1
5:	0000000000000001	0	NOTYPE	GLOBAL	DEFAULT	1	func2

so that when we finally link everything together the linker can map func3 's call back to func2 and also knows where our entry function func3 lives. With our programs stored alongside our data on the computer we're able to automate all of this work to create and run our program. Take this idea further and you'll see that so much of what we take for granted with programs today from their distribution with package managers (https://en.wikipedia.org/wiki/Package_manager) to the very existence of compilers (<https://en.wikipedia.org/wiki/Compiler>) and higher level programming languages (https://en.wikipedia.org/wiki/Programming_language) is because we store our programs alongside our data. Stored program computing for the win!

Segmented Memory

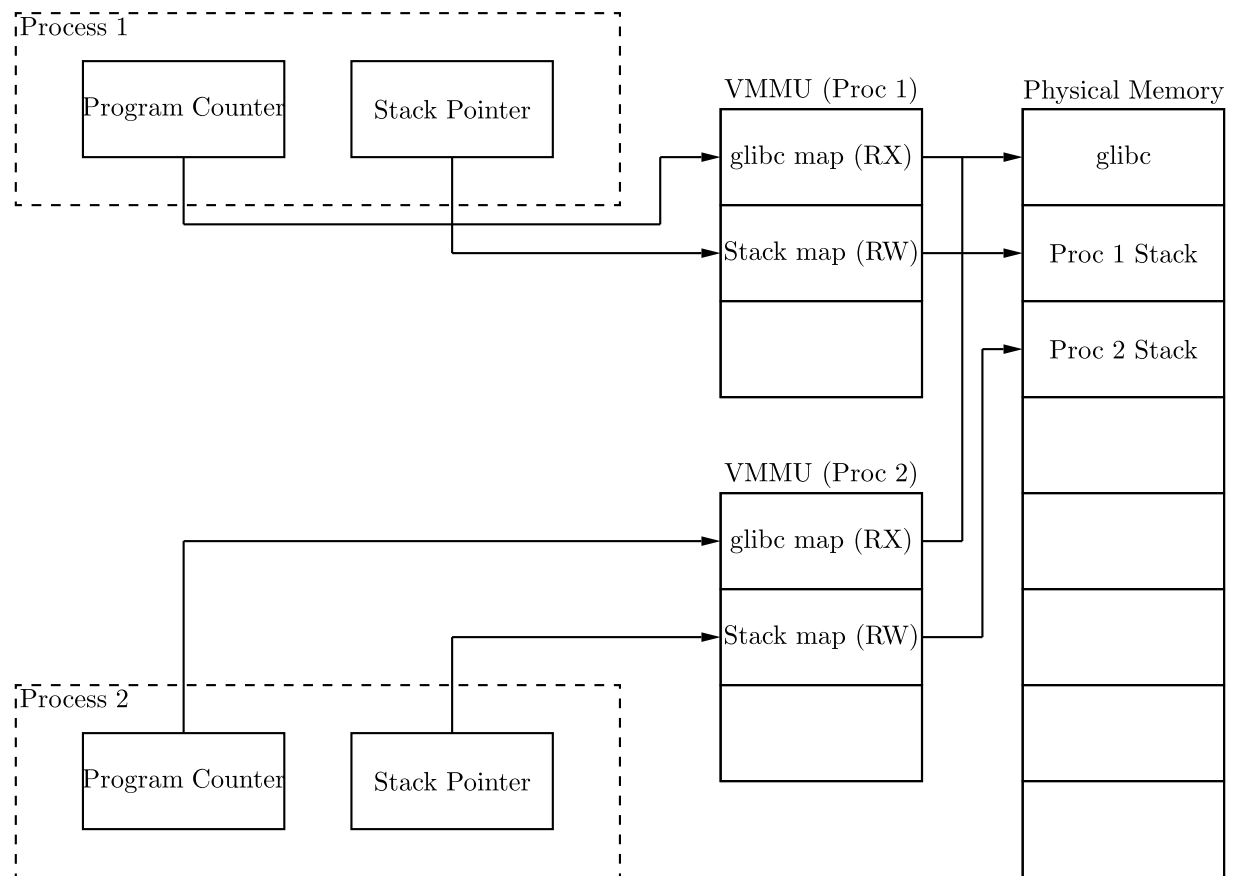
Now you might be wondering what's with the .text at the beginning of the file? Does it do anything or is it just ceremonial? In fact what it does is denote that we are starting a new segment (https://en.wikipedia.org/wiki/Memory_segmentationK) in memory. When we write a program to disk or load ([https://en.wikipedia.org/wiki/Loader_\(computing\)](https://en.wikipedia.org/wiki/Loader_(computing))) it into memory we break it up into these segments. There are a few reasons to do this, but the main one is so that we can prevent our program from being modified once it's running.

See if we make a distinction between the `text` segment that has our instructions and the `data` segment that has our data then we can tell the kernel not to allow any modification to the `text` segment. We might write code that accidentally modifies itself or there could be a malicious hacker who uses a buffer overflow exploit

(https://en.wikipedia.org/wiki/Buffer_overflow) to overwrite our code. In either case locking down our code greatly reduces the odds that bad things happen.

Additionally when your code is read-only you can actually have the same library code shared in memory by multiple processes. The kernel can use virtual memory

(https://en.wikipedia.org/wiki/Virtual_memory) to map multiple processes back to the same read-only segment that it loads once which can save you a ton of memory usage!



Let's write a program that has a data segment to see how gcc does this. How about Hello, world! ? -


```
# hello.s

.text

.global main
main:
    movl    $len,%edx
    movl    $msg,%ecx
    movl    $1,%ebx
    movl    $4,%eax
    int     $0x80

    movl    $0,%ebx
    movl    $1,%eax
    int     $0x80

.data

msg:
    .ascii  "Hello, world!\n"
    len = . - msg
```

As you see we now have a `.text` segment with instructions (which we'll cover in the next section) and a `.data` segment with an `ascii` (<https://en.wikipedia.org/wiki/ASCII>) encoded string. Now let's assemble, link, and take a look at the segments which are stored in the program's program headers (<https://docs.oracle.com/cd/E19683-01/816-1386/chapter6-83432/index.html>).

```
$ gcc -c hello.s -o hello.o
$ ld --entry main hello.o -o hello
$ readelf --program-headers hello
```

Elf file type is EXEC (Executable file)
 Entry point 0x401000
 There are 3 program headers, starting at offset 64

Program Headers:

Type	Offset FileSiz	VirtAddr MemSiz	PhysAddr Flags Align
LOAD	0x0000000000000000 0x00000000000000e8	0x0000000000400000 0x00000000000000e8	0x0000000000400000 R 0x1000
LOAD	0x0000000000000100 0x0000000000000022	0x0000000000401000 0x0000000000000022	0x0000000000401000 R E 0x1000
LOAD	0x0000000000000200 0x000000000000000e	0x0000000000402000 0x000000000000000e	0x0000000000402000 RW 0x1000

Section to Segment mapping:

Segment Sections...

```
00
01 .text
02 .data
```

and you'll see we have three segments. The second one is our text segment and has read and execute permissions (R E) while the third one is our data segment which has read and write permissions (R W). We can confirm our understanding by having our code modify the string in the data segment before printing, say to make it lower case.

```
.global main
main:
    mov     $0x20, %rdi
    mov     $msg,%rsi
    lodsq
    or      %rdi,%rax
    mov     $msg,%rdi
    stosq

    movl    $len,%edx
    movl    $msg,%ecx
    movl    $1,%ebx
    movl    $4,%eax
    int     $0x80

    movl    $0,%ebx
    movl    $1,%eax
    int     $0x80
```

basically we added a few instructions at the beginning to load the first 8 bytes of `msg` into a register, `or` the first byte with `0x20` which in ascii will make it lower case, and then write the results back. If we assemble, link, and run we get

```
$ gcc -c hello.s -o hello.o
$ ld --entry main hello.o -o hello
$ ./hello
hello, world!
```

Awesome. Now let's try to modify our text segment instead

```
# hello.s

.text

.global main
main:
    mov     $main, %rdi
    stosq

    movl    $len,%edx
    movl    $msg,%ecx
    movl    $1,%ebx
    movl    $4,%eax
    int     $0x80

    movl    $0,%ebx
    movl    $1,%eax
    int     $0x80

.data

msg:
    .ascii  "Hello, world!\n"
    len = . - msg
```

You know the drill

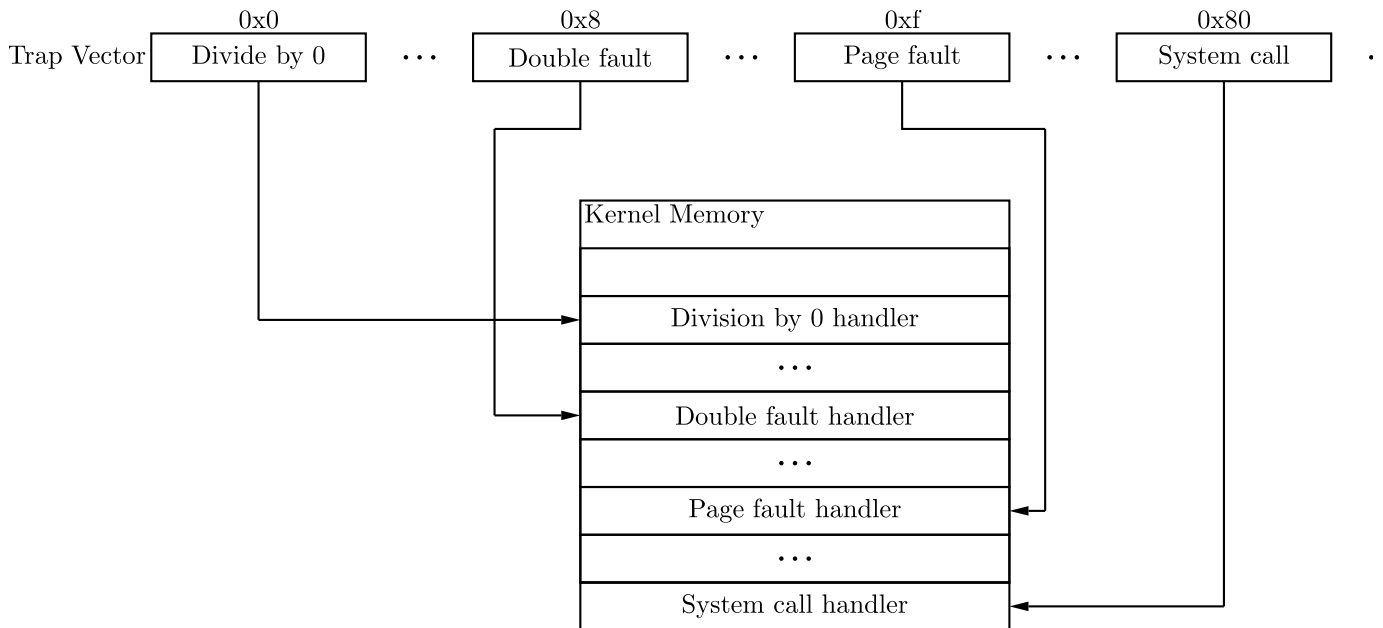
```
$ gcc -c hello.s -o hello.o
$ ld --entry main hello.o -o hello
$ ./hello
Segmentation fault
```

a segfault (https://en.wikipedia.org/wiki/Segmentation_fault), our OS is literally complaining that we are writing to a segment not set for writing. Job done.

Privileged Execution

Finally let's take a close look at how we actually got output from our program. Since doing so ultimately involves hardware access (to our terminal) we get a peek at how we get our OS kernel to do privileged things for us. Obviously we don't want our program to start executing kernel code willie nillie so x86 lets the kernel set up special entry points called **traps** ([https://en.wikipedia.org/wiki/Trap_\(computing\)](https://en.wikipedia.org/wiki/Trap_(computing))), places in memory where it writes its own function to handle special events. The segmentation fault we just saw is one of those trap

events; a page fault would be another such event. Each event type gets its own entry in what's called the trap vector (https://en.wikipedia.org/wiki/Interrupt_descriptor_table), an array that points to all the handlers.



So when we write the assembly code

```
movl    $1,%ebx
movl    $4,%eax
int     $0x80
```

that last instruction `int` will interrupt (<https://en.wikipedia.org/wiki/Interrupt>) the current flow of the program and go to the address of the `0x80` th trap handler. This handler is the special system call (https://en.wikipedia.org/wiki/System_call) handler that a program will use to request that the system do something for it. In this case we are loading the constant 4 into `%eax` which is code for do a write operation (<https://thevivekpandey.github.io/posts/2017-09-25-linux-system-calls.html>) and we load 1 into `%ebx` which is the file descriptor (https://en.wikipedia.org/wiki/File_descriptor) for standard out ([https://en.wikipedia.org/wiki/Standard_streams#Standard_output_\(stdout\)](https://en.wikipedia.org/wiki/Standard_streams#Standard_output_(stdout))).

Let's do a quick experiment to confirm our understanding here. How about we write to standard error instead of standard out? Should be easy enough. The kernel has already set up a file descriptor for us. Just change 1 to 2.

```
movl    $2,%ebx
movl    $4,%eax
int     $0x80
```

we'll redirect the stderr to a file

```
$ gcc -c hello.s -o hello.o
$ ld --entry main hello.o -o hello
$ ./hello 2>/tmp/test
$ cat /tmp/test
Hello, world!
```

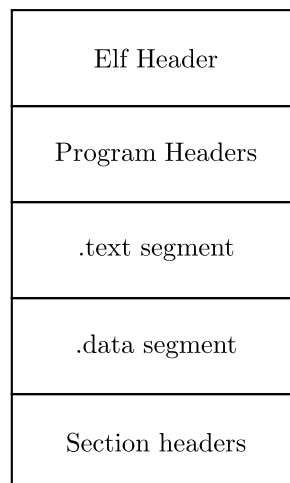
and we see we are indeed writing to stderr. Cool cool.

Part 2: Byte-for-Byte

At this point we've seen most of the functional elements of a binary in action from relocation tables ([https://en.wikipedia.org/wiki/Relocation_\(computing\)](https://en.wikipedia.org/wiki/Relocation_(computing))) to symbol tables (https://en.wikipedia.org/wiki/Symbol_table), segmentation (https://en.wikipedia.org/wiki/Memory_segmentation) and of course the instructions themselves including the interrupt (<https://en.wikipedia.org/wiki/Interrupt>) instruction that supports syscalls (https://en.wikipedia.org/wiki/System_call). In this part we will see how the Executable and Linker Format (ELF) (https://en.wikipedia.org/wiki/Executable_and_Linkable_Format) actually implements these features and represents their information in binary. In each section we'll tamper with the binary slightly to test our understanding.

File Headers

There are 8903 bytes to our program in total, but only 228 non-zero ones we need to understand. Let's start with a grasp of the overall structure.



Basically the file begins with the Elf header, a fixed size series of bytes that gives information about the file overall. Following this you have your program headers table which gives information about your segments and you have the segments themselves.

Each segment has a minimum size of 4KiB because this is the x86 page size (<https://en.wikipedia.org/wiki/Paging>), hence all the zero bytes in our file. Finally we have our symbol table which will tell us the location of our sections and functions.

We get some key information from `readelf`

```
$ readelf --headers hello
ELF Header:
  Magic:   7f 45 4c 46 02 01 01 00 00 00 00 00 00 00 00 00
  ''
  Start of program headers:          64 (bytes into file)
  ...
  Size of program headers:           56 (bytes)
```

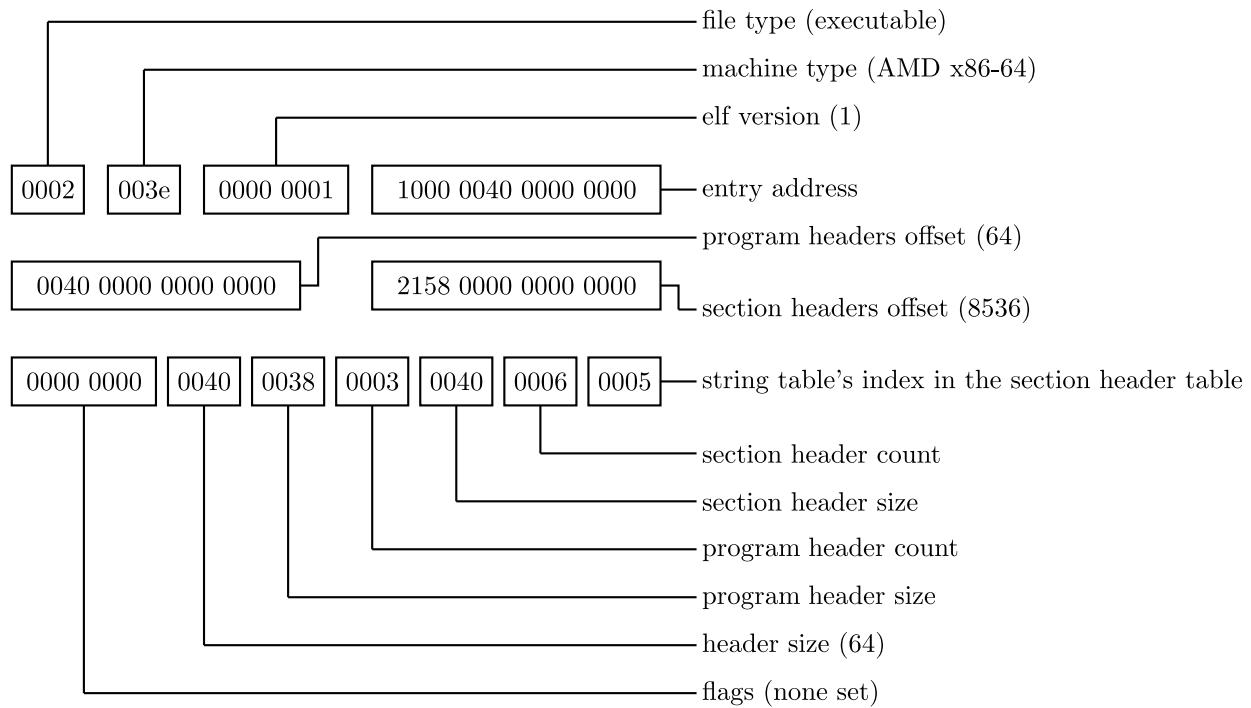
and see for starters that the first 16 bytes are magic numbers

([https://en.wikipedia.org/wiki/Magic_number_\(programming\)](https://en.wikipedia.org/wiki/Magic_number_(programming))). These just indicate that our file is an ELF file so we don't need to break them down further. Our program headers start at byte 64 so that means our ELF header makes up the 48 byte gap. We can see all of it as

```
$ od -x hello | head -4 | tail -3
0000020 0002 003e 0001 0000 1000 0040 0000 0000
0000040 0040 0000 0000 0000 2158 0000 0000 0000
0000060 0000 0000 0040 0038 0003 0040 0006 0005
```

and if we look at the structure of the ELF header

(<https://refspecs.linuxfoundation.org/elf/gabi4+/ch4.eheader.html>) we can break this down as



To test our understanding let's modify our entrypoint. To figure out where we want to jump to we'll disassemble the binary

```
$ objdump -d hello
```

```
hello:      file format elf64-x86-64
```

Disassembly of section .text:

```
0000000000401000 <main>:
401000:      ba 0e 00 00 00      mov     $0xe,%edx
401005:      b9 00 20 40 00      mov     $0x402000,%ecx
40100a:      bb 02 00 00 00      mov     $0x2,%ebx
40100f:      b8 04 00 00 00      mov     $0x4,%eax
401014:      cd 80              int     $0x80
401016:      bb 00 00 00 00      mov     $0x0,%ebx
40101b:      b8 01 00 00 00      mov     $0x1,%eax
401020:      cd 80              int     $0x80
```

and see that if we start at instruction 401016 we'll skip over our write and straight to the exit. So we'll edit just a single entrypoint byte with

```
$ printf '\x16' | dd of=hello bs=1 seek=24 count=1 conv=notrunc
1+0 records in
1+0 records out
1 byte copied, 0.0013276 s, 0.8 kB/s
```


Then confirm the new entrypoint, run the program, and make sure we still exit cleanly.

```
$ readelf --headers hello | grep 'Entry point'
  Entry point address:                0x401016
$ ./hello
$ echo $?
0
```

Sick. Now each program header is 56 bytes and we have three of them. Let's look at just the `.text` segment.

```
$ od -x hello | head -11 | tail -4
0000160 1000 0000 0000 0000 0001 0000 0005 0000
0000200 1000 0000 0000 0000 1000 0040 0000 0000
0000220 1000 0040 0000 0000 0022 0000 0000 0000
0000240 0022 0000 0000 0000 1000 0000 0000 0000
```

We see a couple of instances of `1000 00400` , our slightly mangled address of `0x0040100` so we know we're looking in the right places. Let's see if we can make our `.text` segment writable. If we were hackers this would be a good place to start to exploit the program when it's eventually run.

```
$ printf '\x07' | dd of=hello seek=124 bs=1 count=1 conv=notrunc
1+0 records in
1+0 records out
1 byte copied, 0.0010476 s, 1.0 kB/s
$ readelf --program-headers hello
```

```
Elf file type is EXEC (Executable file)
Entry point 0x401000
There are 3 program headers, starting at offset 64
```

Program Headers:

Type	Offset FileSiz	VirtAddr MemSiz	PhysAddr Flags Align
LOAD	0x0000000000000000 0x00000000000000e8	0x0000000000400000 0x00000000000000e8	0x0000000000400000 R 0x1000
LOAD	0x00000000000001000 0x0000000000000022	0x0000000000401000 0x0000000000000022	0x0000000000401000 RWE 0x1000
LOAD	0x00000000000002000 0x000000000000000e	0x0000000000402000 0x000000000000000e	0x0000000000402000 RW 0x1000

Section to Segment mapping:

```
Segment Sections...
00
01      .text
02      .data
```

and we confirm the text segment is now read-write-executable. We're on our way to becoming black hats, ha.

The Memory Segments

Skipping ahead now because we have a lot of zero bytes we go straight to the start of our `.text` segment at offset `0x1000`. We see ~48 bytes worth of instructions

```
$ od --endian=big -v -x hello | tail -n +257 | head -3
0010000 ba0e 0000 00b9 0020 4000 bb02 0000 00b8
0010020 0400 0000 cd80 bb00 0000 00b8 0100 0000
0010040 cd80 0000 0000 0000 0000 0000 0000 0000
```

which line up exactly with our instructions.

```
$ objdump -d hello
```

```
hello:      file format elf64-x86-64
```

Disassembly of section .text:

```
0000000000401000 <main>:
 401000:      ba 0e 00 00 00      mov     $0xe,%edx
 401005:      b9 00 20 40 00      mov     $0x402000,%ecx
 40100a:      bb 02 00 00 00      mov     $0x2,%ebx
 40100f:      b8 04 00 00 00      mov     $0x4,%eax
 401014:      cd 80                int     $0x80
 401016:      bb 00 00 00 00      mov     $0x0,%ebx
 40101b:      b8 01 00 00 00      mov     $0x1,%eax
 401020:      cd 80                int     $0x80
```

Pretty straightforward. Skipping next to the `.data` segment we see unsurprisingly our "Hello, world!" string

```
$ od --endian=big -v -x hello | tail -n +513 | head -2
0020000 4865 6c6c 6f2c 2077 6f72 6c64 210a 0000
0020020 0000 0000 0000 0000 0000 0000 0000 0000
```

We can use python to confirm

```
$ python -c 'print(list(map(hex, map(ord, "Hello, world!"))))'
['0x48', '0x65', '0x6c', '0x6c', '0x6f', '0x2c', '0x20', '0x77', '0x6f', '0x72']
```

The Symbol Table

The last part of our binary which we will examine in depth will be its symbol table. As we've seen, this table encodes a mapping from symbolic names like `msg` and `main` to their offsets in the object file. As a refresher, let's see what this table looks like for our "Hello, world!" binary.

```
$ readelf --symbols hello
```

Symbol table '.symtab' contains 10 entries:

Num:	Value	Size	Type	Bind	Vis	Ndx	Name
0:	0000000000000000	0	NOTYPE	LOCAL	DEFAULT	UND	
1:	0000000000401000	0	SECTION	LOCAL	DEFAULT	1	
2:	0000000000402000	0	SECTION	LOCAL	DEFAULT	2	
3:	0000000000000000	0	FILE	LOCAL	DEFAULT	ABS	hello.o
4:	000000000000000e	0	NOTYPE	LOCAL	DEFAULT	ABS	len
5:	0000000000402000	0	NOTYPE	LOCAL	DEFAULT	2	msg
6:	000000000040200e	0	NOTYPE	GLOBAL	DEFAULT	2	__bss_start
7:	0000000000401000	0	NOTYPE	GLOBAL	DEFAULT	1	main
8:	000000000040200e	0	NOTYPE	GLOBAL	DEFAULT	2	_edata
9:	0000000000402010	0	NOTYPE	GLOBAL	DEFAULT	2	_end

Taking a look at the structure of the symbol table (https://docs.oracle.com/cd/E23824_01/html/819-0690/chapter6-79797.html) we can see how each of these elements is encoded. Each entry is 24 bytes and from inspecting our file we can see that the entries begin at offset 0x2010 . Using our nify link from before we can decompose each entry to tell us the associated address, where in the string table the symbol's name starts, and whether the symbol is local or global. Let's write a python script to do this for us,

```
# elf_parse.py

with open("hello", 'rb') as f:
    elf = f.read()

table_start = 0x2010
count = 10
entry_len = 24
string_table_start = 0x2100

for i in range(count):
    entry_start = table_start + i * entry_len
    entry = elf[entry_start: entry_start + entry_len]
    addr = reversed(entry[8:16])
    addr_s = "".join((hex(byt)[2:].zfill(2) for byt in addr))
    info = entry[4]
    bind = "GLOBAL" if info & 0b00010000 else "LOCAL "
    name_offset = string_table_start + int.from_bytes(entry[:4], byteorder='li')
    name_chars = []
    while elf[name_offset] != 0:
        name_chars.append(chr(elf[name_offset]))
        name_offset += 1
    name = ''.join(name_chars)
    print("{}\t{}\t{}".format(addr_s, bind, name))
```

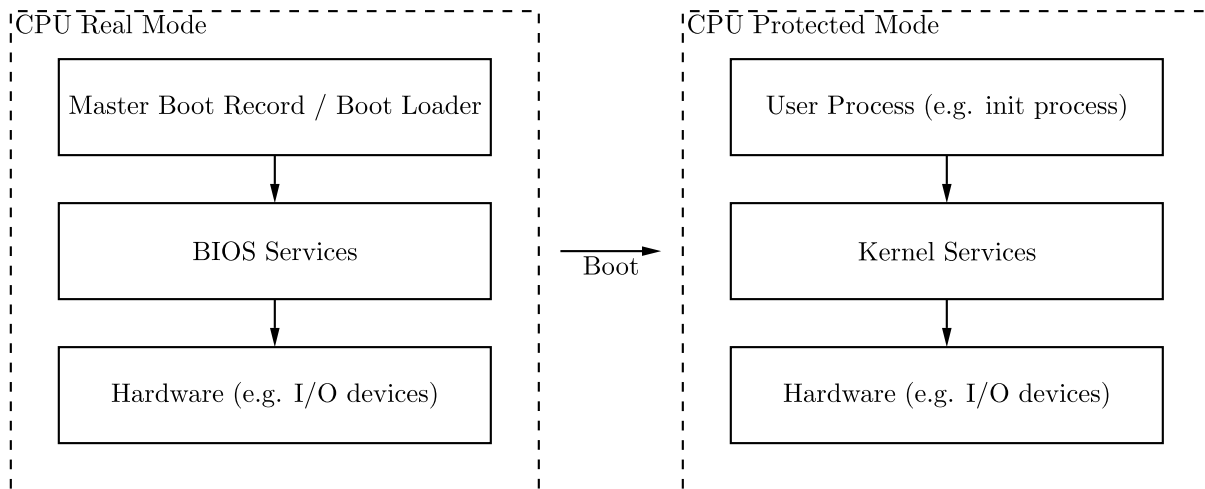
which when we run we see the exact same info that `readelf` gave us.

```
$ python3 elf_parse.py
0000000000000000      LOCAL
0000000000040100      LOCAL
0000000000040200      LOCAL
0000000000000000      LOCAL  hello.o
000000000000000e      LOCAL  len
0000000000040200      LOCAL  msg
000000000004020e      GLOBAL  __bss_start
0000000000040100      GLOBAL  main
000000000004020e      GLOBAL  _edata
0000000000040210      GLOBAL  _end
```

So looks like we've understood the structure correctly. At this point we've analyzed and understood just about every byte in our program. It has been a slog, but hopefully you have taken away more than just the bytes of `Hello, world!` and come to understand the general principles at work when you assemble code.

Bonus Part 3: Making it Bootable

In our last (optional) part we're going to go one layer deeper down the rabbit hole and see what a program looks like when it doesn't have an operating system supporting it. That means no virtual memory and no system calls! Your computer is only in such a state very briefly when it needs to pull the operating system up by its own bootstraps (hence, we call it "booting up").



During this time an x86 CPU will be in "real" mode which means that memory addresses don't go through the VMMU we saw before but correspond to real addresses of your physical memory. To interact with I/O peripherals you still use an interrupt, but in this case you're going to a trap set up by the system BIOS (<https://en.wikipedia.org/wiki/BIOS>) which will be loaded from a (possibly read-only (https://en.wikipedia.org/wiki/Read-only_memory)) memory chip on your motherboard.

Without much fanfare let's see what our final assembly will look like

```
# hello_bootable.s

.code16
.global boot
boot:
    mov $0xe, %ah
    mov $msg, %si

print_si:
    lodsb
    int $0x10
    cmp $0, %al
    jne print_si

halt:
    hlt

msg:
    .ascii "Hello world!"
    len = . - msg

.fill 510-(.-boot), 1, 0
.word 0xaa55
```

A few things to note:

- First we'll mark this file with `.code16` to tell our assembler to write instructions for a 16-bit real-mode CPU. To ensure compatibility with the original 8086 (https://en.wikipedia.org/wiki/Intel_8086) processor, all x86 processors start in this mode.
- Similarly we are not segmenting our memory here because we have no facilities for segmented memory. Both our instructions and our data get crammed in the same area of memory that is readable, writable, and executable.
- Instead of interrupting to trap `0x80` we are using trap `0x10` which is the BIOS interrupt handler (https://en.wikipedia.org/wiki/INT_10H). Among other things it can write out text for us, one character at a time. The `0xe` in register `%ah` tells the handler we want to write the byte that was loaded by `lodsrb` into register `%si`.
- Finally you'll see we are zero padding the end of our binary to fill 510 bytes and then adding `0xaa55` at the end.

This last bit requires a little explaining. See what an x86 system will do to boot from a device is load the first 512 bytes of that device, called the boot sector

(https://en.wikipedia.org/wiki/Boot_sector), into memory at address `0x7c00`

(https://en.wikipedia.org/wiki/Master_boot_record#Programming_considerations) ('cuz other stuff needs

the addresses before that one). This code loaded from the boot sector may then go and directly initialize an operating system in which case we call it a bootloader (<https://en.wikipedia.org/wiki/Booting#Boot-loader>) or it may select a partition from the boot device and load the bootloader from there in which case it is called a master boot record (https://en.wikipedia.org/wiki/Master_boot_record). Either way x86 will perform a sanity check on this sector and ensure it ends with the magic bytes `0xaa55` denoting a bootable sector.

Let's assemble as before to produce our object file. Then we'll link with a few special options to produce a bootable binary

```
$ gcc -c hello_bootable.s -o hello_bootable.o
$ ld --oformat binary -e boot -Ttext 0x7c00 -o hello_bootable hello_bootable.o
```

First we specify the output format as `binary` so that the produced file is the raw sequence of bytes to load into memory (no elf headers, no relocation tables, no nothing). Also we're going to specify `0x7c00` as the address at which our bytes will start when loaded. Without this the linker doesn't know how to resolve symbolic names like `$msg` to physical addresses. Let's inspect the file we produced. First we'll confirm it is 512 bytes long.

```
$ wc hello_bootable
0 2 512 hello_bootable
```

Check. Let's make sure it ends with the right magic number

```
$ od -x hello_bootable
00000000 0eb4 0dbe ac7c 10cd 003c f975 48f4 6c65
00000020 6f6c 7720 726f 646c 0021 0000 0000 0000
00000040 0000 0000 0000 0000 0000 0000 0000 0000
*
00000760 0000 0000 0000 0000 0000 0000 0000 aa55
00010000
```

Ok. And finally let's see what instructions we produced.


```
$ objdump -b binary -mi8086 -D hello_bootable
```

```
hello_bootable:      file format binary
```

```
Disassembly of section .data:
```

```
00000000 <.data>:
 0:  b4 0e                mov     $0xe,%ah
 2:  be 0d 7c             mov     $0x7c0d,%si
 5:  ac                  lods    %ds:(%si),%al
 6:  cd 10                int     $0x10
 8:  3c 00                cmp     $0x0,%al
 a:  75 f9                jne     0x5
 c:  f4                  hlt
 d:  48                  dec     %ax
 e:  65 6c              gs insb (%dx),%es:(%di)
10:  6c                  insb    (%dx),%es:(%di)
11:  6f                  outsw   %ds:(%si),(%dx)
12:  20 77 6f            and     %dh,0x6f(%bx)
15:  72 6c                jb      0x83
17:  64 21 00            and     %ax,%fs:(%bx,%si)
   ...
1fe:  55                  push    %bp
1ff:  aa                  stos    %al,%es:(%di)
```

Note here that we have to explicitly tell `objdump` not to look for any headers with `-b binary` and that the file is for 16-bit x86 with `-mi8086`. We can confirm here that our second instruction has resolved a static address for the string of `0x7c0d`. Checks out. Everything after instruction `c` is meaningless as that's our ascii string, but `objdump` can't tell the difference.

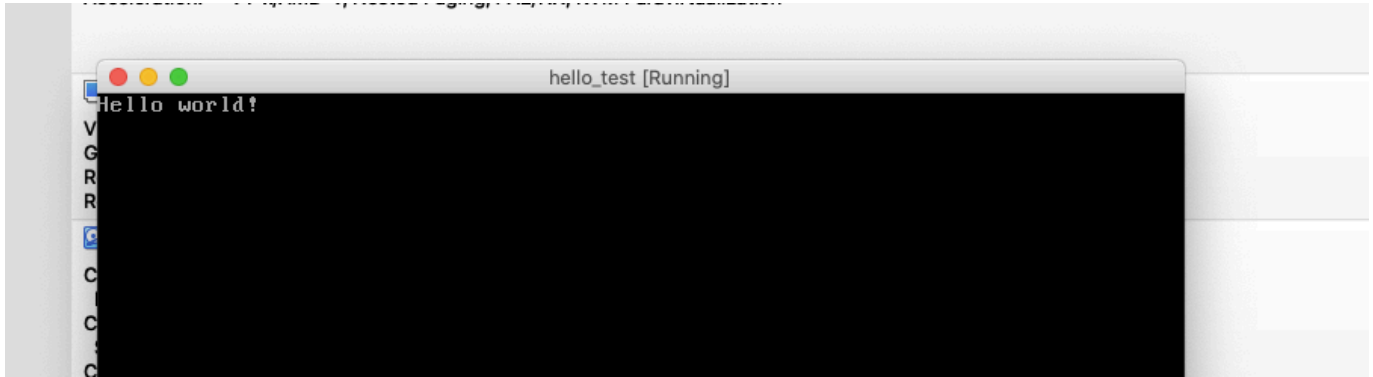
Finally we'll have to write these bootable bytes to a "disk" – in our case a virtual floppy disk image file. We'll start by initializing our file with 1474560 zero bytes, precisely the 1.44Mb of a floppy disk.

```
$ dd if=/dev/zero of=hello.flp bs=1474560 count=1
1+0 records in
1+0 records out
1474560 bytes (1.5 MB, 1.4 MiB) copied, 0.014878 s, 99.1 MB/s
```

Next we'll copy our boot sector to the front of the file, being careful not to remove the rest

```
$ dd if=hello_bootable of=hello.flp conv=notrunc  
1+0 records in  
1+0 records out  
512 bytes copied, 0.00115951 s, 442 kB/s
```

Finally if we attach this floppy disk to a new virtual machine in Virtual Box we see with glee our Hello, world! boot message.



Hello world indeed. Until next time!

Further Reading

- ELF Man Page (<https://man7.org/linux/man-pages/man5/elf.5.html>), particularly for decoding headers
- Linkers and Loaders (https://www.amazon.com/Linkers-Kaufmann-Software-Engineering-Programming/dp/1558604960/ref=sr_1_2)
- Learning Linux Binary Analysis (https://www.amazon.com/Learning-Binary-Analysis-elfmaster-ONeill/dp/1782167102/ref=sr_1_1_ssapa)
- The Linux Programming Interface (https://www.amazon.com/Linux-Programming-Interface-System-Handbook/dp/1593272200/ref=sr_1_1), in particular Ch. 2 - 4, 6, 7, 41 - 42, 49 - 50